

Roles, areas and guard rails

Review of the admin and vendor accounts (2026-09-25): who can open what, what was wrong, and what now stops it.

1. The accounts

Role	Users (local)	Permissions	Where they belong
Administrator	1	133, including both <code>dashboard_access</code> (staff) and <code>dashboard_vendor_access</code> (vendor)	the admin area, and the vendor area for their own business
Vendor	4	46: catalogue create/edit for each service type, coupons, news, enquiries, media, <code>dashboard_vendor_access</code>	the vendor area
Customer	4	none	their own account pages
Custom staff roles	none yet	anything made in Role Manager, for example staff with <code>dashboard_access</code> but no vendor access	the admin area only

The administrator role holds both sets of permissions, so the default administrator never meets the problems below. They appear as soon as a staff role without a vendor account exists, and for vendors and customers who wander.

2. What was wrong

1. **The vendor area had no role gate.** A customer with zero permissions could open TourPay, the statement, API keys, integrations, analytics, Tanova, the wallet and payouts. Data stayed scoped to their own user id (nobody saw another business's data), but they could create API keys and invoices with no vendor account. Some pages crashed instead (`/user/dashboard` , `/user/tour` , `/vendor/payouts`).
2. **Turned away in silence.** A vendor opening an admin page was bounced to their dashboard with no explanation; a stray link could double-redirect through `/admin` .
3. **Staff without a vendor account got errors.** The "Vendor Dashboard" link and Payouts returned a 500, with no way back.
4. **Admin menu entries that dropped staff into the vendor shell.** Tanova opens the vendor layout; the only way back was an "Admin Dashboard" link at the very bottom of the scrolling vendor sidebar. The header had no link.
5. **TourPay in the admin menu crashed** (500), and its detail page would have used the vendor layout.
6. `/admin/integrations` and the platform legal documents were open to any signed-in person, so a vendor could rewrite the platform's Terms and Privacy Policy (fixed earlier today).
7. **18 admin controllers had no per-permission check** (wallet credit, plan requests, modules, tools, e-mail and SMS tests, statistics, TourPay, availability calendars, template live editor). A limited staff role could reach them.

3. The guard rails now

One map, `config/areas.php`, read by one middleware, `AreaGuard`, and checked by a test.

Area	Who	Permission
staff	the platform team	<code>dashboard_access</code>
vendor	a business	<code>dashboard_vendor_access</code>
account	any signed-in person (profile, own bookings, wallet, security)	none

- **Turned away, never sent in a circle.** Every redirect goes somewhere that person can open: vendor or customer on a staff page goes home; staff without a vendor account on a vendor page goes to `/admin`; a customer on a vendor page goes to their profile. Each carries a visible note saying why. Requests that expect JSON get a 403.
- **Never the upgrade action as a landing page.** `/user/upgrade-vendor` files the vendor request when opened, so customers are sent to their profile, where "Become a vendor" is a deliberate click.
- **Two-way switcher** for people who have both: a "Vendor view" button in the admin header, and a "← Admin" button in the vendor header, always visible.
- **Extra staff permissions** for screens with no check of their own: wallet credit and plan requests need `user_update`; modules, tools, e-mail and SMS tests need `setting_update`; statistics and the credit report need `report_view`; the template live editor needs `template_update`; each availability calendar needs that service's `_update`; TourPay needs `tourpay_view`.
- **Staff TourPay** is now a read-only view across all businesses, in the admin layout (list with totals per currency, filters, and a detail page).
- **Deny by default.** A test lists every admin, user and vendor page and fails if one is not placed in an area, so a new screen cannot ship without someone deciding who it is for.

4. Access matrix (verified by tests)

Page	Administrator	Staff only	Vendor	Customer
<code>/admin</code> , admin modules	open	open	to their dashboard, with a note	to their profile, with a note
<code>/admin/integrations</code> and legals	open	open	to their dashboard	to their profile
Vendor dashboard, TourPay, API keys	open	to <code>/admin</code> , with a note	open	to their profile, with a note
Profile, own bookings, wallet	open	open	open	open
Staff sections needing an extra permission	open	only with that permission	n/a	n/a

5. Still to decide or do

- **GET `/user/upgrade-vendor` changes data** (creates the request, and can assign the vendor role when auto-approval is on). It should be a POST with a form button.
- **Staff two-factor.** Two-factor sign-in exists but is optional; consider requiring it for staff and for anyone with payout or wallet permissions.

- **Vendor team members.** Confirm what a team member may do inside the owner's business; today it is the vendor role's permissions.
- **Audit trail for staff actions** (wallet credit, plan approvals, module changes): the money ledger and TourPay have one; the admin area does not.
- **Re-check custom roles** when one is created: the extra-permission map covers the screens listed above; other admin screens still rely on the checks inside their own controllers (59 of 78 have them).